Invisible Work

Measuring the Human Cost of AI Alignment

George Abrahamyants
J.D., University of California, Hastings College of the Law
B.A., University of California, Berkeley

February 2026

Author's Note

I read your January 28 paper, “Who’s in Charge? Disempowerment Patterns in Real-World LLM Usage,” and was struck by the gap you call out around safeguards that only operate at the single-exchange level. I built a tool to measure the other side of that gap: the human effort required to keep models on task over many turns. It computes an Operator Load Index and related metrics (Alignment Tax, Instruction Survival Rate, Self-Sufficiency Score) from real transcripts, using a 10-tag model-side taxonomy and a 12-rule operator-side system. It was run on 512 of my own Claude and ChatGPT conversations (37,536 messages), and the results show similar drift scores but very different operator load profiles—one model fails loudly and high-maintenance, the other fails quietly with much lower visible intervention.

The write-up is attached; the code and live demo are in the Drift Auditor repository here: https://github.com/gugosf114/drift-auditor. If this is useful for your internal evaluations or external research work, you’re welcome to use or adapt any part of it.

The Gap

On January 28, 2026, Anthropic published "Who's in Charge? Disempowerment Patterns in Real-World LLM Usage," analyzing 1.5 million Claude.ai conversations for patterns where AI responses undermine a user's independent judgment. The paper identifies a specific limitation in current safety infrastructure:

> *"Our current safeguards operate primarily at the individual exchange level, which means they may miss behaviors like disempowerment potential that emerge across exchanges and over time."1*

This matched what I had been observing for months. Single-turn evaluation sees a model acknowledge a correction and concludes the problem is fixed. Multi-turn analysis reveals the acknowledgment changed nothing. The model said the right thing, then went back to doing the wrong thing, and nobody noticed because nobody was measuring across turns.

Separately, I read the Opus 4.5 system card[2] and examined Anthropic's two open-source auditing tools: Petri, which explores model behavior through simulated multi-turn conversations and surfaces unexpected patterns[3]; and Bloom, which generates targeted evaluation suites to measure how often a specific behavior appears across models.[4] Both tools were used in Anthropic's system card evaluations.

Petri asks: *does this behavior exist?* Bloom asks: *how often does it occur?* The disempowerment paper asks: *did the model's output undermine the user's judgment?* All three evaluate the same side of the interaction: what the model did.

None of them ask the complementary question: **what did the human have to do to prevent it?**

A model that produces clean outputs because the operator corrected it 741 times is not safe. It is high-maintenance. A model that produces clean outputs with 10 corrections is actually safe. Current evaluations cannot distinguish between these two cases. The metric that would distinguish them does not exist yet. That is what this paper introduces.

How I Got Here

Drift Auditor is the endpoint of eight stages. Each one produced something useful and a specific failure that forced the next stage. I document these because the research community treats tools as arrivals. This one is a trail of documented dead ends.

Stage	What I Tried	What Broke
1	Better prompts	Model complies initially, drifts after ~15 turns
2	Meta-rules (12 Rules for AI) system prompt	Rules followed, then silently dropped
3	Sandboxes 10-Layer	Layers enforced locally, not across turns
4	NotebookLM digests	Description of drift, no detection capability
5	Mid-chat debugging (Iron Pipeline)	Debugging inherits the drift it tries to measure
6	Binary detection	Drift/No-Drift is insufficient; need severity + type
7	Multi-class detection	Still contaminated by generator context
8	Air-gapped architecture	Current tool (heuristic weights not calibrated)

The critical discovery came at Stage 5. When you debug a conversation using the same model that produced the conversation, the debugger inherits the same priors that caused the drift. It produces false negatives because it is already contaminated. The auditor must be air-gapped from the generator. It cannot see the generator's context. This is the architectural principle that separates Drift Auditor from prompt-based debugging, and it was discovered by failing at every simpler approach first.

Stage 2 produced "12 Rules for AI: An Operator's Field Manual" (29 pages, 17 academic references), which documents the operator heuristics that became the 12-rule classification system in Drift Auditor.[5] Stage 3 produced the AEGIS 10-Layer Architecture, whose void detection layer (Given → Acknowledged → Followed → Persisted) became the per-instruction lifecycle tracking in the current tool.[6]

What the Tool Does

Drift Auditor classifies both sides of a conversation. It tags what the model did and what the human did. No existing tool does the second part.

The Model Side: 10-Tag Taxonomy

Tag	Category	What It Detects
SYCOPHANCY	Commission	Unsolicited praise, invented agreement
REALITY_DISTORT	Commission	False references, hallucinated context
CONF_INFLATE	Commission	Unwarranted certainty without evidence
INSTR_DROP	Omission	Silently stops following an instruction
SEM_DILUTE	Omission	Follows instruction in letter, not spirit
CORR_DECAY	Persistence	Acknowledged correction does not persist
CONFLICT_PAIR	Structural	Automatable contradiction between turns
SHADOW_PATTERN	Structural	Emergent behavior not prompted by operator
OP_MOVE	Operator	Audit of human's steering action
VOID_DETECTED	Meta	Break in the causal chain

Tags 1–6 extend existing detection categories. Tags 7–10 are new. CONFLICT_PAIR finds contradictions the model made against itself across turns. SHADOW_PATTERN catches behaviors nobody asked for. OP_MOVE—the operator tag—has no equivalent anywhere. It classifies what the human did, not what the model did.

The Operator Side: 12-Rule System

Derived from the 12 Rules for AI field manual and 250+ adversarial conversations. Each rule classifies a specific type of corrective action the operator took:

Rule	Name	What the Operator Did
R01	Anchor	Set explicit instruction at conversation start
R02	Echo Check	Asked model to restate instructions
R03	Boundary	Enforced scope limits (most frequently violated)
R04	Correction	Direct error correction
R05	Not-Shot	Caught voice transcription / typo errors

R06	Contrastive	"What changed between X and Y?"
R07	Reset	Started over / full context reset
R08	Decompose	Broke complex instruction into steps
R09	Evidence Demand	"Show me where" / proof request
R10	Meta Call	Called out the drift pattern by name
R11	Tiger Tamer	Active reinforcement—kept correcting
R12	Kill Switch	Abandoned thread / hard stop

Detection Methods

Twenty-plus detection methods organized in four layers. Layer 1 catches commission errors (sycophancy, reality distortion, confidence inflation). Layer 2 catches omissions using sentence-transformer embeddings to measure semantic drift between instructions and responses. Layer 3 tracks whether corrections actually hold across subsequent turns. Layer 4 classifies epistemic posture (RED/YELLOW/GREEN) using surface markers.

The advanced methods include contrastive anchoring (diff early vs. late instruction presence), void detection (the Given → Acknowledged → Followed → Persisted chain), edge-vs-middle positional analysis, pre-drift signals (four early warning indicators), and coupling score—a measure of how much downstream reasoning depends on each instruction. If the instruction vanished, would the rest of the conversation change? HIGH means yes.

Adversarial Test Generator

The tool also generates test conversations automatically. One model gets a system prompt with specific instructions. A second model plays the user, pushing on those instructions for 50+ turns. The resulting transcript gets audited by the Drift Auditor engine. Five pre-built scenarios: hedging persistence, citation decay, format compliance, boundary respect, correction persistence. This parallels Anthropic's Bloom pipeline[4] with the addition of operator-side classification.

The Dataset

512 real conversations. 37,536 messages. 187 from Claude, 325 from ChatGPT. All involving one operator—me—over approximately six months. These are not synthetic benchmarks. They are real working conversations: code generation, compliance analysis, creative writing, research, adversarial testing. Conversations under 10 messages were excluded.

The single-operator design is a limitation and a control. Same person, same correction patterns, same expectations, both models. Any difference in operator load is attributable to the model, not the operator.

What I Found

Metric	Claude	ChatGPT
Conversations analyzed	187	325
Total messages	20,407	17,129
Average drift score	2.9 / 10	3.0 / 10
Operator Load Index	0.082	0.056
Alignment Tax	16.3%	11.2%
Self-Sufficiency Score	59.2%	72.1%
Instruction Survival Rate	56.5%	45.3%
Correction Efficiency	98.7%	100.0%
Total corrections issued	741	101
Corrections that failed	10	0

The headline drift scores are nearly identical. 2.9 versus 3.0. If you stopped there—if you used a single metric the way most evaluations do—you would conclude these models perform equivalently on alignment.

They do not.

Claude is **high-maintenance, high-fidelity**. It triggered 741 corrections. It costs 16.3% of the conversation just keeping it on track. It requires 1.5x more intervention than ChatGPT. But when you give it an instruction, it retains 56.5% of them. When you correct it, the correction holds 98.7% of the time. It fails loudly. You notice.

ChatGPT is **low-maintenance, silent decay**. It runs more autonomously. It triggers fewer corrections. It has perfect correction efficiency—when corrected, it holds 100% of the time. But it silently drops 55% of instructions without the operator ever noticing. The low intervention rate is not evidence of alignment. It is evidence that the failure mode is invisible.

One fails loudly. The other fails quietly. The quiet one is more dangerous.

This connects directly to the disempowerment research. The paper notes that "users tend to perceive potentially disempowering exchanges favorably in the moment."[1] Silent decay is the mechanism. A model that drops your instructions without triggering corrections creates the feeling of smooth interaction while progressively deviating from what you actually asked for. You rate the conversation highly because you did not notice the drift. You did not notice the drift because the model did not make you work to fix it. That is what Operator Load measures.

Adversarial Results

First automated test: Claude Sonnet on the hedging_persistence scenario. Instructed to be direct and avoid hedging for 60 turns. Scored 6/10 drift in 19 turns. Auto-generated contradiction at turn 10. Full transcripts and reports for five scenarios are in the repository.[7]

Limitations

Scoring weights are heuristic, not empirically calibrated against a ground-truth dataset of confirmed drift events. The sentence-transformer embeddings used for semantic omission detection (all-MiniLM-L6-v2) are a lightweight proxy, not a deep semantic analysis. Layer 4 barometer patterns are surface markers, not actual model uncertainty. The single-operator design controls for variance but limits generalizability to other operator styles and correction patterns. Raw operator load numbers are not directly comparable across models when the operator uses different models for different task complexities; normalized operator load corrected for task complexity is the metric that matters, and that normalization is not yet automated.

References

1. Anthropic et al., "Who's in Charge? Disempowerment Patterns in Real-World LLM Usage." January 28, 2026. anthropic.com/research/disempowerment-patterns
2. Anthropic, "Claude Opus 4.5 System Card." 2025. docs.anthropic.com/en/docs/resources/model-card
3. Fronsdal, K., Gupta, I., Sheshadri, A., et al., "Petri: An Open-Source Auditing Tool to Accelerate AI Safety Research." Anthropic, 2025. anthropic.com/research/petri-open-source-auditing
4. Gupta, I., Fronsdal, K., Sheshadri, A., et al., "Bloom: An Open Source Tool for Automated Behavioral Evaluations." Anthropic, 2025. anthropic.com/research/bloom
5. Abrahamyants, G., "12 Rules for AI: An Operator's Field Manual." 29 pages, github.com/gugosf114/12-rules-for-ai
6. Abrahamyants, G., "AEGIS 10-Layer Architecture." Audit Ledger project. github.com/gugosf114/audit-ledger
7. Drift Auditor repository, adversarial test results and batch data. github.com/gugosf114/drift-auditor

Appendix: Linked Resources

Resource	Location
Drift Auditor (source + data)	github.com/gugosf114/drift-auditor
Live demo	drift-auditor-3vghngnafvgkpqwdditewy.streamlit.app
Batch results (512 conversations)	/batch_results/ and /batch_results_chatgpt/
Operator Load comparison	/operator_load_comparison.txt
Adversarial transcripts	/adversarial_results/
12 Rules → Tool mapping	/docs/research/12_rules_mapping.md
12 Rules for AI (full paper)	github.com/gugosf114/12-rules-for-ai
Audit Ledger (Newton system)	github.com/gugosf114/audit-ledger
Bakers Agent	github.com/gugosf114/bakers-agent